

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO – UFRSA

Centro de Ciências Exatas e Naturais – CCEN

Departamento de Computação - DC

Graduação em Ciência da Computação

Disciplina: Estrutura de Dados II

Prof.: Paulo Henrique Lopes Silva

Prática Offline 3

1. Cache Eviction.

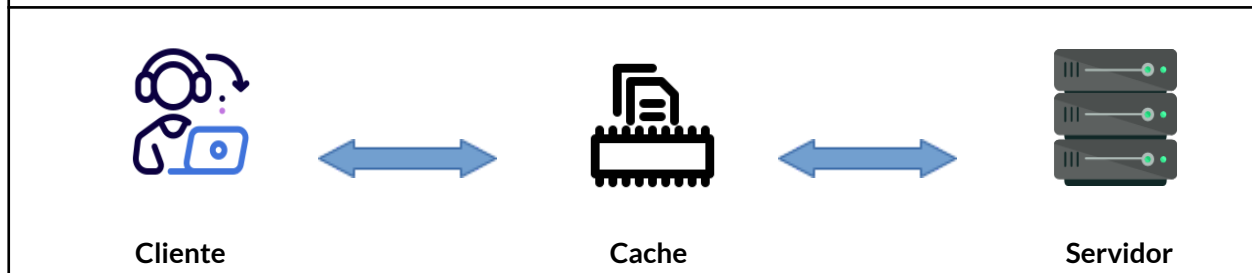
- **Cache eviction** é o processo de remoção de itens do cache quando ele atinge sua capacidade máxima, a fim de liberar espaço para novos dados.
- Esse processo é essencial para manter a eficiência do cache, evitando que ele fique cheio e incapaz de armazenar novos dados.

2. Considere a necessidade do desenvolvimento de uma base de dados sobre ordens de serviço (OS) para reparar falhas em equipamentos.

- Tarefa:
 - **Simulação de uma aplicação cliente/servidor, onde:**
 - Uma classe ou conjunto de classes implementa o cliente que pode fazer buscas na base de dados;
 - Uma classe ou conjunto de classes implementa o servidor que controla a base de dados.
 - **Mensagens:**
 - Uma mensagem é um objeto que encapsula os atributos de uma OS, a operação que vai ser feita e alguma outra opção que você queira protocolar nesta comunicação.
 - As mensagens entre cliente e servidor devem ser “enviadas” usando **compressão de dados**.
 - Cliente **comprime** antes de enviar a mensagem e servidor **descomprime** após receber a mensagem.
 - Utilize o algoritmo de Huffman.
 - Use um algoritmo de processamento de strings para verificar os caracteres e suas frequências nas mensagens.
 - Use lista de prioridade como estrutura auxiliar durante a compressão.
 - Não é permitido usar estruturas já prontas.
 - **Ordens de serviço:**
 - Cada ordem de serviço possui dados como código, nome, descrição e hora da solicitação.
 - **Sobre a cache e a base de dados:**
 - A base de dados deve ser implementada como uma **Tabela Hash**.
 - A modelagem da tabela hash (função de dispersão e tratamento de colisões) faz parte da interpretação do projeto (use orientação a objetos). Justifique as escolhas.
 - Se a tabela hash usar o tratamento de colisões com encadeamento exterior, use uma **lista autoajustável** como lista de colisão.

- É importante que a base de dados seja uma tabela hash redimensionável.
 - A cache pode ser implementada como uma **lista de prioridades** ou uma **lista autoajustável** para 30 elementos e deve utilizar:
 - A cache só deve ser preenchida a partir de buscas.
 - Crie uma função para fazer um número de buscas que preencha a cache.
 - Operação de busca na cache.
 - Buscas devem ser feitas primeiro na cache. Caso o item de busca não esteja na cache, a busca é redirecionada para a base de dados.
 - Inserções, alterações e remoções devem ser feitas na base de dados e refletidas na cache.
 - Política para esvaziamento da cache: vai depender da estrutura que for escolhida. Justifique a escolha.
 - Justifique a escolha da função de dispersão e, eventualmente, do mecanismo de tratamento de colisões para as tabelas da base de dados.
- **Requisitos da simulação:**
- Tanto o cliente como o servidor vão ser executados no mesmo computador (simulados).
 - Entre o cliente e o servidor existe a cache.
 - Clientes podem fazer consultas (**buscas**) usando o código.
 - Outras operações que um cliente pode fazer são:
 - **Cadastrar OS.**
 - **Listar** ordens de serviço (todos os dados de todas as ordens de serviço).
 - **Alterar OS.**
 - **Remover OS.**
 - **Acessar a quantidade** de registros.
 - Servidor atende às requisições do cliente.
 - O servidor deve manter uma espécie de log informando o estado da tabela hash a cada operação (sugiro inserir essas informações em um arquivo).
 - Além disso, deve mostrar os itens da cache a cada operação.
 - Para mostrar a política de **cache eviction** funcionando.
 - Um cliente pode fazer a autenticação para acessar a aplicação?
 - É facultativo.
 - Tratamento de erros e exceções onde for possível (interpretativo).
- **Execução da simulação:**
- Inicia o programa:
 - 100 ordens de serviço devem ser adicionadas na base de dados do servidor.
 - Realizar (crie funções para as operações):
 - Três consultas.
 - Uma listagem.
 - Dois cadastros seguidos de uma listagem.
 - Duas alterações seguidas de uma listagem.
 - Três remoções seguidas de uma listagem.
 - Escolha operações de forma a evidenciar o funcionamento do **cache eviction**.
 - Buscar as ocorrências de operações como cadastrar e remover registros de OS no servidor.
 - Use um algoritmo de processamento de strings.

3. Representação Gráfica da Simulação.



4. Avaliação:

- O trabalho vale 100% da nota da 3ª unidade.
- Deve ser desenvolvido na linguagem Java usando orientação a objetos.
 - Data de entrega: 23/10/2024, somente via sigaa.
 - Formato de envio: seu-nome-pratica-off-3.zip.
- **O projeto é individual.**
 - Em caso de alguma dúvida, o(a) aluno(a) poderá ser chamado para esclarecimentos.
- **Em caso de comprovação de cópia ou fraude, os trabalhos envolvidos receberão a nota ZERO.**
- Entregou fora do prazo?
 - O desconto na nota será de 20%.
 - Data limite: 24/10/2024.
 - Entregas não serão aceitas após a data limite.